

EIF209 Programación 4

Examen parcial #2

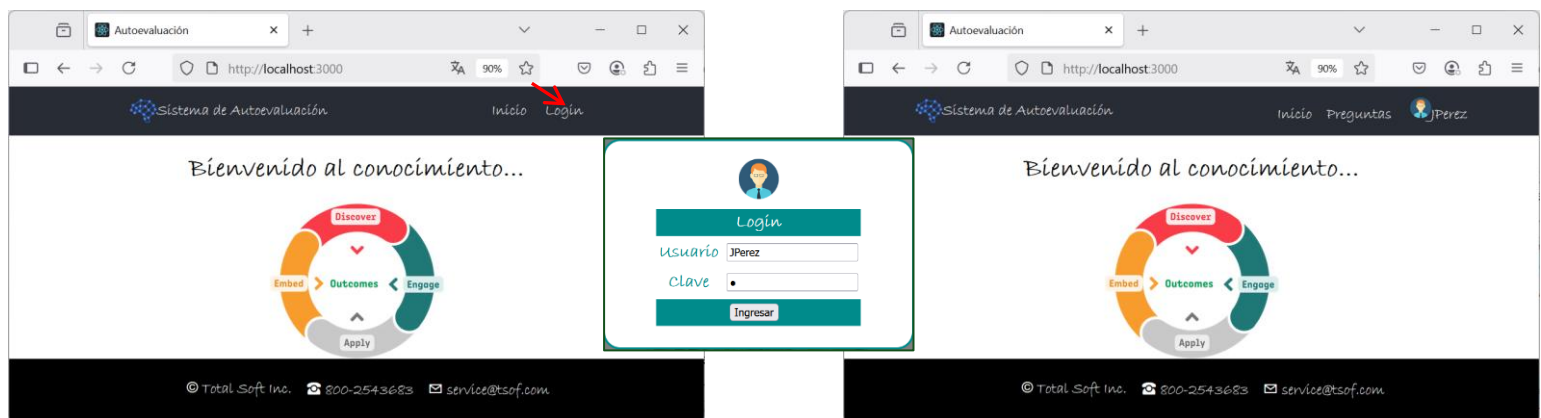
Duración: 4 hrs

El examen debe ser resuelto en modalidad presencial en los laboratorios de la Escuela de Informática. No se permite el uso de internet ni de teléfonos celulares.

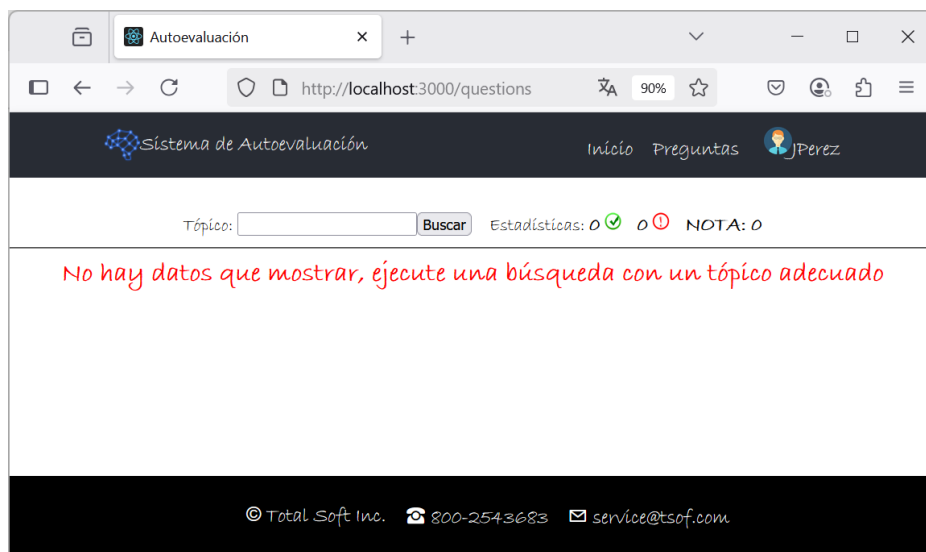
Deberá construir parte de un sitio *web* para un “**Autoevaluación**” de conocimientos, bajo la modalidad de SPA (Single Page Application o aplicación de una sola página). La aplicación permitirá seleccionar preguntas de una base pre-existente y responder cualquiera de ellas. **Cada pregunta tendrá varias opciones, de ellas algunas serán correctas. Una respuesta del usuario se considera correcta si selecciona cualquiera de las opciones correctas de la pregunta.** El sistema llevará registro de las respuestas del usuario y de su estadística de aciertos y fallos. Cada pregunta respondida por el usuario ya no estará disponible en las búsquedas para ese usuario, pero por supuesto que sí lo estará para otros usuarios que aún no la hayan respondido.

El **FrontEnd** (interfaz de usuario) deberá ser desarrollado usando renderizado del lado del cliente, aplicando el framework **React** (u otro similar basado en **Javascript**). **Solo se permite usar los paquetes estándar de React: *react, react-dom, react-router-dom*.** El **BackEnd** (lógica de la aplicación) deberá desarrollarse usando servicios **RESTful** con el framework **Spring**. Los datos se mantendrán en memoria, usando una base de datos en memoria como **H2**, u otro mecanismo. La comunicación del FrontEnd con el BackEnd se hará por medio de peticiones asíncronas, usando un API como **Fetch** u otro similar. La seguridad (Autenticación y Autorización) deberá implementarse usando **JWT** (Json Web Token) y estará basada en el rol del usuario que ingrese.

Al cargar la página ésta mostrará el componente de Inicio, mostrando también un enlace para ingresar (Login). Una vez que un usuario ingrese se mostrará enlaces a **Inicio**, a **Preguntas**, y uno para **salir** (con el nombre del usuario). La funcionalidad para ingresar (**Login**) puede implementarse con una ventana emergente (popup).

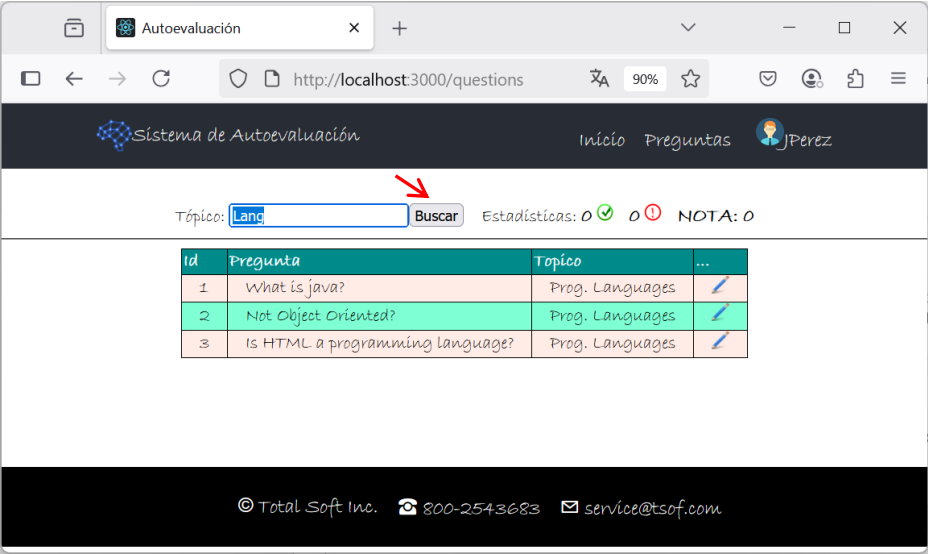


El componente (página) de **Preguntas** es como se muestra en la siguiente imagen.

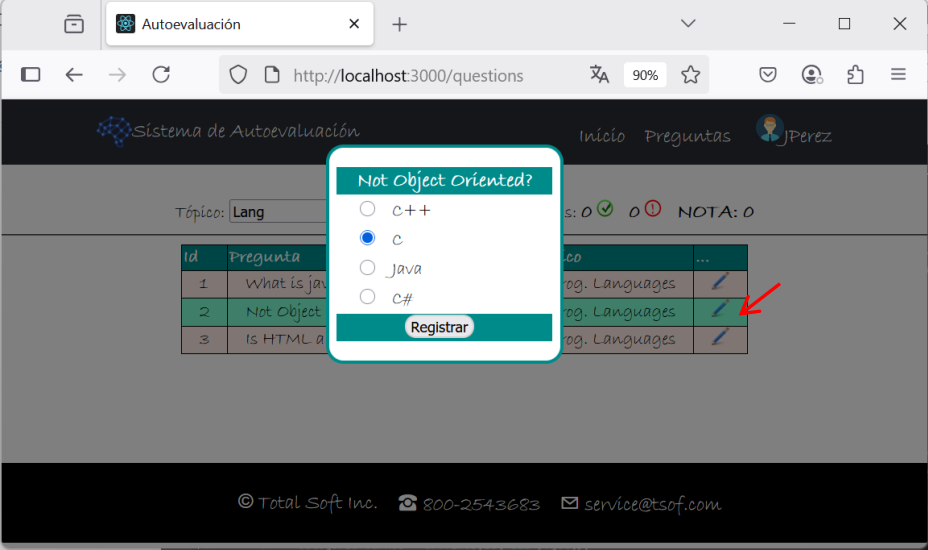


Habrá una facilidad de búsqueda por aproximación al tópico de la pregunta. También habrá estadísticas de aciertos (✓), fallas (⚠) y nota. Si ya el usuario hubiera contestado preguntas en sesiones anteriores sus estadísticas reflejarían los datos correspondientes. Cuando el usuario haga una búsqueda la lista de preguntas aparecerá en pantalla. Al inicio, al no haber preguntas se mostrará el mensaje indicado.

Al hacer una búsqueda, las preguntas que correspondan y que aún no hayan sido respondidas por el usuario se mostrarán en pantalla. **En este paso solo deben traerse del servidor las preguntas, no las opciones de cada una.**

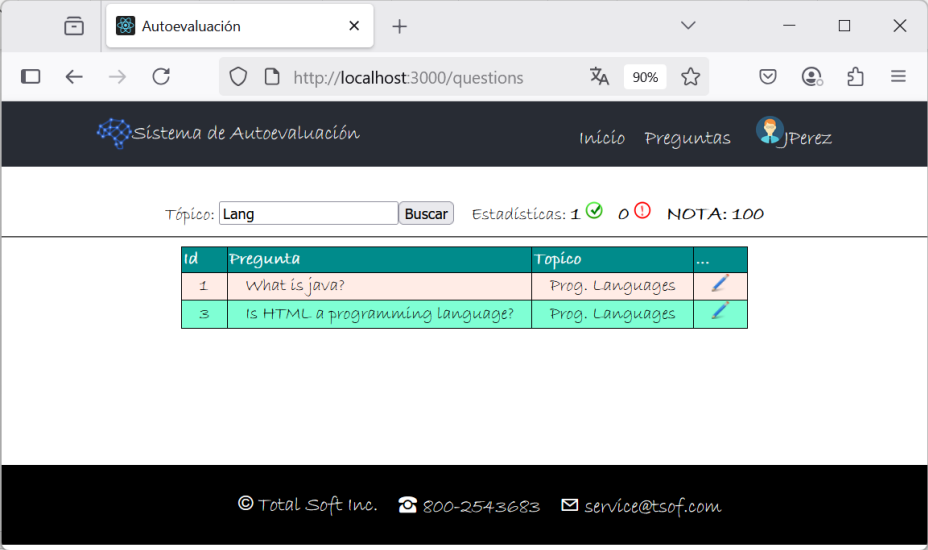


Cuando el usuario seleccione una pregunta (✎) se debe mostrar una ventana emergente (“popup”) con los datos de la pregunta y las opciones de respuesta, de manera que el usuario pueda seleccionar una.



La información sobre cuál es la opción correcta para una pregunta nunca será traída al *frontend*. Solo se usará en el *backend* para determinar si la respuesta del usuario fue correcta o no.

Cuando el usuario registre su respuesta, dicha información será enviada al servidor (*backend*) para ser registrada. En pantalla (*frontend*) se refrescará la lista de preguntas, pues la que fue contestada ya no estará disponible para ese usuario, y se actualizarán las estadísticas.



El ejercicio se calificará de acuerdo con la siguiente tabla de puntuación:

Punto de evaluación	Pts.
Página inicial (componente de Inicio)	5
Funcionalidad de ingresar (Login)	10
Mostrar Estadísticas	15
Hacer Búsqueda y mostrar lista de preguntas	20
Mostrar una pregunta y sus opciones mediante la ventana emergente	25
Responder la pregunta, registrándola en el servidor y actualizando la pantalla	25

Datos de Referencia.

Solo para efectos de referencia, las siguientes imágenes muestran algunos datos que se usaron para fines ilustrativos. Estas imágenes, así como el correspondiente código provisto, son solo de referencia.

```
© DataLoader.java x
18 public DataLoader(UserRepository userRepository, PreguntaRepository preguntaRepository, OpcionRepository opcionRepository) {
19     BCryptPasswordEncoder bCryptPasswordEncoder = new BCryptPasswordEncoder();
20     userRepository.save(new User( id: "JPerez", bCryptPasswordEncoder.encode( rawPassword: "1"), rol: "CLI", name: "Juan Perez"));
21     userRepository.save(new User( id: "MMata", bCryptPasswordEncoder.encode( rawPassword: "1"), rol: "CLI", name: "Maria Mata"));
22
23     Pregunta p; Opcion op1,op2,op3,op4;
24     p=new Pregunta( pregunta: "What is java?", topico: "Prog. Languages");
25     op1=new Opcion("A programming language");
26     op2=new Opcion("A scripting language");
27     op3=new Opcion("A animation language");
28     opcionRepository.saveAll(Arrays.asList(op1, op2, op3));
29     p.setOpciones(Arrays.asList(op1, op2, op3));
30     p.setCorrectas(Arrays.asList(op1));
31     preguntaRepository.save(p);
32
33     p=new Pregunta( pregunta: "A sql keyword", topico: "Data Bases");
34     op1=new Opcion("Insert");
35     op2=new Opcion("New");
36     op3=new Opcion("Update");
37     opcionRepository.saveAll(Arrays.asList(op1, op2, op3));
38     p.setOpciones(Arrays.asList(op1, op2, op3));
39     p.setCorrectas(Arrays.asList(op1,op3));
40     preguntaRepository.save(p);
```